

SUMMER 2026



LONGHORN HAPKIT

Lab guide #2 - morning

ROSS NEUMAN

Assistant Professor of Practice, The University of Texas at Austin

<https://github.com/rneum/RobotCamp2026>

Kinematics for handle

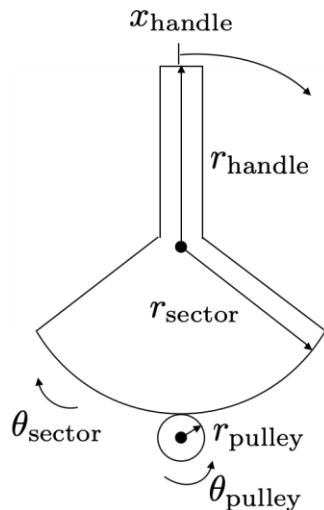
- Recall the kinematics discussed
- Write code that will report the position x_{handle}

$$- r_{\text{pulley}} = 0.5 \text{ cm}$$

$$- r_{\text{sector}} = 7.5 \text{ cm}$$

$$- r_{\text{handle}} = 8.5 \text{ cm}$$

$$- \theta_{\text{pulley}} = \text{encoder reading in radians}$$



$$r_{\text{pulley}} \theta_{\text{pulley}} = r_{\text{sector}} \theta_{\text{sector}}$$

$$x_{\text{handle}} = r_{\text{handle}} \theta_{\text{sector}}$$



$$x_{\text{handle}} = \frac{r_{\text{handle}} r_{\text{pulley}}}{r_{\text{sector}}} \theta_{\text{pulley}}$$

Kinematics for handle

- Code should look something like this
- x_h is the position of the handle in cm

```
// read encoder
pos = encoder.read();

tPulley = (pos / encoderResolution) * 2 * PI; //angular position of motor pulley in radians

tS = (tPulley * rp) / rs; //angular position of sector pulley in radians

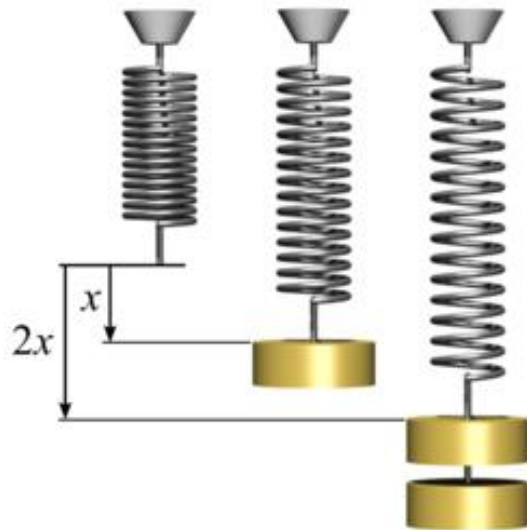
//determine handle position in cm
xh = (rh * rp * tPulley) / rs;
```

Now we will use this position to inform:

- A virtual spring
- A virtual wall

Virtual Spring

- $F_{spring} = -k * x_{handle}$
- This is also known as “Hooke’s Law”
- “k” is your spring constant, or stiffness
- You should play around with different values of k to see what the limits of your hapkit feel like
- F_{spring} , for our purposes, will just be the command sent to the motor
 - This is because we are equating our voltage command to the motor with the amount of force produced
 - This is not correct, but works okay for this situation

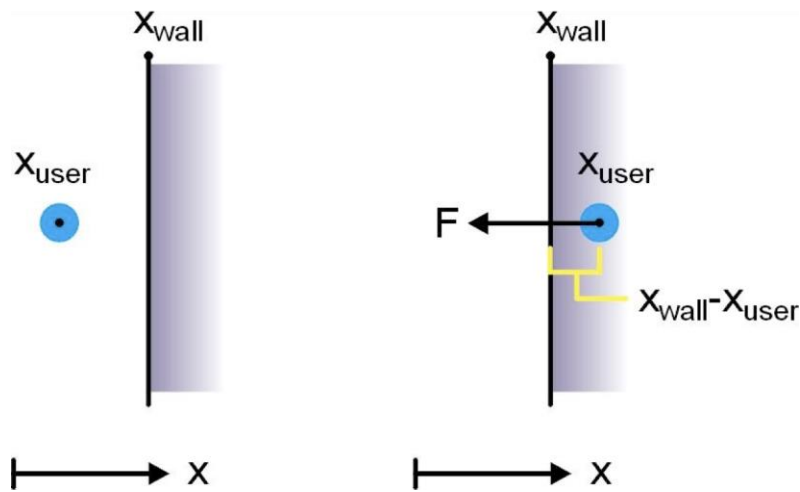


```
const float k = 20.0;  
motorCommand = -k * xh;
```

Create a Virtual Wall

- Decide on a location for your wall
- Write a conditional statement (if/else) that will command a very high stiffness once you are “inside” the wall

$$F = \begin{cases} -k(x - x_{wall}) & x > x_{wall} \\ 0 & x \leq x_{wall} \end{cases}$$

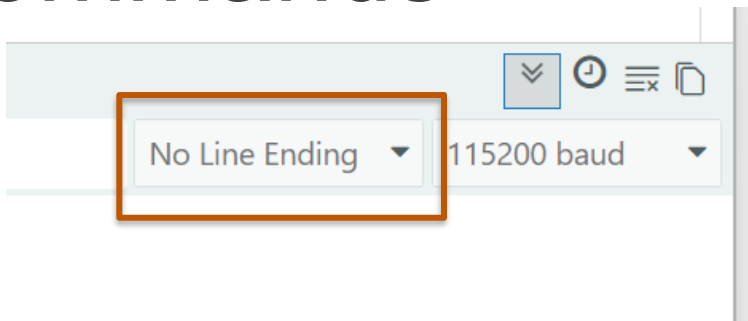


With your virtual wall

- Try moving the wall around
 - Place at +3cm
 - Place at -2cm
 - Place two virtual walls simultaneously

Use serial to send commands

- Serial can also be used to send commands, not just read out what the program is offering
- `Serial.read()` tells the serial monitor to read commands the user is entering when available
- Set serial monitor to “no line ending”
- Create a new variable called “mode” that you will use to tell the program which haptic environment to render
 - Put this at the top where you are declaring other variables like pos, xh, etc.
- Now in `loop()` you can read for the mode
- Continued on next page...



With variable declarations:

```
// Selected virtual environment  
char mode = '0';
```

In `loop()`:

```
// Update mode if a serial command arrives  
if (Serial.available()) {  
  mode = Serial.read();  
}
```

Commanding Modes

- The code shown to the side implements a *switch statement* that allows you to switch between your spring and wall mode without re-uploading the code
- You can add more cases to this statement as you develop more virtual environments if you like this workflow better

```
// Update mode if a serial command arrives
if (Serial.available()) {
  mode = Serial.read();
}

// Compute force continuously
switch (mode) {

  // No movement
  case '0':
  {
    motorCommand = 0;
    break;
  }

  // Virtual spring
  case '1':
  {
    const float k = 20.0;
    motorCommand = -k * xh;
    break;
  }

  // Virtual wall
  case '2':
  {
    const float x_wall = 2.0;
    const float k_wall = 200.0;

    if (xh > x_wall) {
      motorCommand = -k_wall * (xh - x_wall);
    } else {
      motorCommand = 0;
    }
    break;
  }
}

// Send command to motor, cast as an integer
setMotor((int)motorCommand);
```

Make a zeroing function

- Another convenient function is to have one that resets the encoder to 0
 - If your capstan slips at all, use this to re-establish your $x = 0$ point
- Simply use `encoder.write()` to tell it the position is now 0
- I like to make this my “0” function in the switch statement

```
// No movement/ zeroing
case '0':
{
    motorCommand = 0;
    encoder.write(0);
    pos = 0;
    break;
}
```

Make a simple “bump”

- Place a “bump” at a desired location
- Play around with the parameters (width, force, location)
- Try creating multiple bumps
- See if you can design a way to make this bump feel more realistic

```
//Bump
case '3':
{
    float bump_location = 2;
    float bump_width = 1;
    float bump_force = 150;
    if (abs(xh - bump_location) < bump_width / 2) {
        motorCommand = -bump_force;
    } else {
        motorCommand = 0;
    }

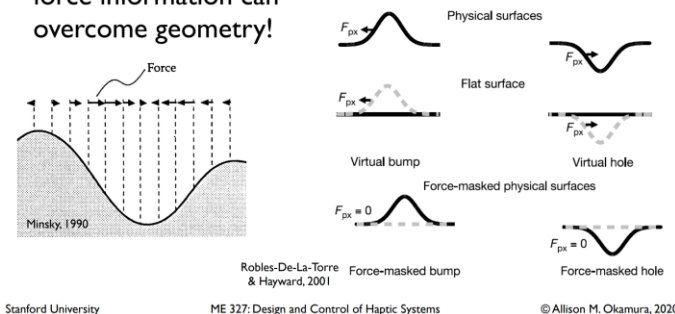
    break;
}
```

More realistic bump

- A real bump will push against you on the way “up” and then help you along on the way “down”
- Turn this bump into a valley instead

bumps and valleys

- as a user moves “up” a bump, motion is opposed.
- done in 2D, spring force proportional to height of bump
- force information can overcome geometry!



```
float bump_location = 2;
float bump_width = 1;
float bump_force = 150;

if ((xh > bump_location - bump_width / 2) && (xh < bump_location)) {
    motorCommand = -bump_force;
}

else if ((xh >= bump_location) && (xh < bump_location + bump_width / 2)) {
    motorCommand = bump_force;
}

else{
    motorCommand = 0;
}
```

Bump and Valley – Continued Improvement

- For an extra challenge--can you make it feel like a smooth hill instead?
 - Could use sinusoids
 - Could use piecewise functions

Continue to experiment with Composing virtual environments

- Get creative, see what you can make!